Denoising Diffusion Probabilistic Models and Generative Adversarial Networks

From VAEs and Diffusion Models to GANs — A Unified Derivation

Morten Hjorth-Jensen

Department of Physics & CCSE, University of Oslo

May 4-8, 2026

Outline I

Outline II

Outline III

Generative models: the common thread

All deep generative models share one goal: learn a distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x})$ over data $\boldsymbol{x}$ that we can both evaluate and sample from.

Family	**Latent**	**Training signal**	**Sample cost**
VAE	$\boldsymbol{h} \in \mathbb{R}^{d_h}$, $d_h \ll d_x$	ELBO	1 decoder pass
Diffusion	$\boldsymbol{x}_t \in \mathbb{R}^{d_x}$ (full dim)	**ELBO,** T **levels**	T denoiser calls
GAN	$\boldsymbol{z} \in \mathbb{R}^{k}$	Adversarial	1 generator pass

Key insight

Diffusion models are **hierarchical VAEs** with T latent levels, a fixed (non-learned) encoder, and full-dimensional latents. Understanding VAEs is the fastest route to understanding diffusion.

The VAE: ELBO and reparameterisation (recap)

Goal. Maximise $\log p_{\boldsymbol{\theta}}(\boldsymbol{x})$ — intractable because $p(\boldsymbol{x}) = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{h})\, p(\boldsymbol{h})\, d\boldsymbol{h}$.

Introduce approximate posterior $q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})$. By Jensen's inequality on $\log$:

$$\log p(\boldsymbol{x}) = \log \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})}\left[\frac{p_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{h})}{q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})}\right] \;\geq\; \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})}\left[\log \frac{p_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{h})}{q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})}\right] =: \mathcal{L}(\boldsymbol{\phi},\boldsymbol{\theta}).$$

Splitting the joint $p(\boldsymbol{x},\boldsymbol{h}) = p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{h})\, p(\boldsymbol{h})$:

$$\boxed{\mathcal{L}(\boldsymbol{\phi},\boldsymbol{\theta}) = \underbrace{\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{h})]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}\big(q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x}) \,\|\, p(\boldsymbol{h})\big)}_{\text{regularisation}}}$$

Reparameterisation. With Gaussian encoder $q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{\phi}}, \boldsymbol{\sigma}^2_{\boldsymbol{\phi}}\mathbf{I})$ and prior $p(\boldsymbol{h}) = \mathcal{N}(\mathbf{0},\mathbf{I})$:

$$\boldsymbol{h} = \boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}) + \boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0},\mathbf{I}).$$

Closed-form KL: $D_{\mathrm{KL}}(q\|p) = \frac{1}{2}\sum_j \Big(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\Big)$.

Why VAEs produce blurry samples

The ℓ_2-averaging problem. The decoder minimises

$$-\mathbb{E}_{q_\phi}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{h})] \;\propto\; \mathbb{E}_{q_\phi}\left[\|\boldsymbol{x} - \hat{\boldsymbol{x}}_{\boldsymbol{\theta}}(\boldsymbol{h})\|^2\right].$$

Because the posterior $q_\phi(\boldsymbol{h}|\boldsymbol{x})$ has nonzero variance, $\hat{\boldsymbol{x}}_{\boldsymbol{\theta}}$ learns the *mean over all likely completions*, which is systematically blurry.

Other VAE limitations

- **Posterior collapse**: powerful decoder can ignore $\boldsymbol{h}$, KL$\to 0$, latent useless.
- **ELBO gap**: $\log p(\boldsymbol{x}) = \mathcal{L} + D_{\mathrm{KL}}(q_\phi \| p(\boldsymbol{h}|\boldsymbol{x})) > \mathcal{L}$.
- Mean-field Gaussian approximation may miss multi-modal true posteriors.

The diffusion solution

- Replace 1 latent level with $T \gg 1$.
- Fix the encoder: no collapse possible.
- Predict only the *noise added at one step*, not the full image — eliminates ℓ_2-averaging artefacts.

Markovian hierarchical VAE (MHVAE)

Generalise VAE to a **chain of T latent spaces** $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T$ (we already write $\boldsymbol{x}_t$ for the t-th latent):

$$p(\boldsymbol{x}_0, \boldsymbol{x}_{1:T}) = p(\boldsymbol{x}_T)\, p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1) \prod_{t=2}^{T} p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) \quad \text{(generative)}$$

$$q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0) = q(\boldsymbol{x}_1|\boldsymbol{x}_0) \prod_{t=2}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) \quad \text{(inference)}$$

Substituting into the ELBO and using the Markov property:

$$\log p(\boldsymbol{x}_0) \geq \mathbb{E}_{q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0)}\left[\log \frac{p(\boldsymbol{x}_T)\, p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1) \prod_{t=2}^{T} p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)}{q(\boldsymbol{x}_1|\boldsymbol{x}_0) \prod_{t=2}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})}\right]$$

Each level contributes a reconstruction/regularisation pair.

Three constraints that define a diffusion model

A **Variational Diffusion Model** (VDM; Kingma et al. 2021; Luo 2022) is an MHVAE with three additional constraints:

Constraint 1 — Full-dimensional latents

$\dim(\boldsymbol{x}_t) = \dim(\boldsymbol{x}_0)$ for all t. No bottleneck; the latents live in *data space*.

Constraint 2 — Fixed Gaussian encoder (forward process)

$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = \mathcal{N}\big(\boldsymbol{x}_t;\, \sqrt{\alpha_t}\,\boldsymbol{x}_{t-1},\, (1-\alpha_t)\mathbf{I}\big), \quad \alpha_t = 1 - \beta_t \in (0,1).$$

Not trained. The noise schedule $\{\beta_t\}$ is a design choice.

Constraint 3 — Gaussian terminal distribution

$\{\alpha_t\}$ is chosen so that $q(\boldsymbol{x}_T|\boldsymbol{x}_0) \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$ as $t \to T$. The model “forgets” the data completely.

Forward process: the Gaussian product rule

Key tool. If $\boldsymbol{x} = a\boldsymbol{u} + b\boldsymbol{v}$ with $\boldsymbol{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $\boldsymbol{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ independent, then

$$\boldsymbol{x} \sim \mathcal{N}\big(\mathbf{0},\ (a^2 + b^2)\mathbf{I}\big).$$

Proof of closed-form marginal. Write one step as $\boldsymbol{x}_t = \sqrt{\alpha_t}\,\boldsymbol{x}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_{t-1}$ with $\boldsymbol{\epsilon}_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Unrolling two steps:

$$\begin{aligned}
\boldsymbol{x}_t &= \sqrt{\alpha_t}\Big(\sqrt{\alpha_{t-1}}\,\boldsymbol{x}_{t-2} + \sqrt{1-\alpha_{t-1}}\,\boldsymbol{\epsilon}_{t-2}\Big) + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_{t-1} \\
&= \sqrt{\alpha_t\alpha_{t-1}}\,\boldsymbol{x}_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\,\boldsymbol{\epsilon}_{t-2} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_{t-1}}_{\sim\mathcal{N}(\mathbf{0},\,[\alpha_t(1-\alpha_{t-1})+(1-\alpha_t)]\mathbf{I})} \\
&= \sqrt{\alpha_t\alpha_{t-1}}\,\boldsymbol{x}_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\,\bar{\boldsymbol{\epsilon}}
\end{aligned}$$

since $\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\alpha_{t-1}$.

Closed-form forward marginal $q(\boldsymbol{x}_t|\boldsymbol{x}_0)$

Define $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. Continuing the induction:

Closed-form marginal (key result)

$$\boxed{q(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}\big(\boldsymbol{x}_t;\ \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I}\big)}$$

Equivalently, we can sample $\boldsymbol{x}_t$ *directly* in one step:

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Interpretation.

- $\sqrt{\bar{\alpha}_t}$ is the *signal coefficient*: decays toward 0.
- $\sqrt{1-\bar{\alpha}_t}$ is the *noise coefficient*: grows toward 1.
- At $t = 0$: $\boldsymbol{x}_0 = \boldsymbol{x}_0$ (clean).
- At $t = T$: $\boldsymbol{x}_T \approx \boldsymbol{\epsilon}$ (pure noise).

No forward-pass through a network needed — this is just arithmetic.

Signal-to-noise ratio and noise schedules

Define the **signal-to-noise ratio**:

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}.$$

$\mathrm{SNR}(0) \gg 1$ (clean data); $\mathrm{SNR}(T) \approx 0$ (pure noise). The VDM objective can be rewritten entirely in terms of $\mathrm{SNR}(t)$, and the loss weight of each term equals $\mathrm{SNR}(t) - \mathrm{SNR}(t+1)$ (Kingma et al. 2021).

Linear schedule (Ho et al. 2020):

$$\beta_t = \beta_{\min} + \tfrac{t-1}{T-1}(\beta_{\max} - \beta_{\min})$$

$\beta_{\min} = 10^{-4}$, $\beta_{\max} = 0.02$, $T = 1000$.

Near-zero SNR early, then rapid collapse.

Cosine schedule (Nichol & Dhariwal 2021):

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos^2\!\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right)$$

$s = 0.008$. SNR decreases more gradually, improving training at low t.

The cosine schedule avoids the very small β_t values that waste early training steps on near-identical images.

Tractable forward posterior $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$

The *unconditional* reverse $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ is intractable. But conditioned on $\boldsymbol{x}_0$ it is tractable via Bayes' rule:

$$q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) = \frac{q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \boldsymbol{x}_0)\, q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_0)}{q(\boldsymbol{x}_t|\boldsymbol{x}_0)} = \frac{q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})\, q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_0)}{q(\boldsymbol{x}_t|\boldsymbol{x}_0)}.$$

The last equality uses the Markov property: $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \boldsymbol{x}_0) = q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$.

Each factor is Gaussian:

$$\begin{aligned} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) &= \mathcal{N}(\sqrt{\alpha_t}\,\boldsymbol{x}_{t-1},\ (1-\alpha_t)\mathbf{I}) \\ q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_0) &= \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}}\,\boldsymbol{x}_0,\ (1-\bar{\alpha}_{t-1})\mathbf{I}) \\ q(\boldsymbol{x}_t|\boldsymbol{x}_0) &= \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I}) \end{aligned}$$

Substituting and completing the square in $\boldsymbol{x}_{t-1}$ yields a Gaussian in $\boldsymbol{x}_{t-1}$.

Forward posterior: completing the square

The log of the numerator (as a function of $\boldsymbol{x}_{t-1}$) is:

$$-\frac{(\boldsymbol{x}_t - \sqrt{\alpha_t}\,\boldsymbol{x}_{t-1})^2}{2(1-\alpha_t)} - \frac{(\boldsymbol{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\,\boldsymbol{x}_0)^2}{2(1-\bar{\alpha}_{t-1})}$$

$$= -\frac{1}{2}\left[\frac{\alpha_t}{1-\alpha_t}\boldsymbol{x}_{t-1}^2 - \frac{2\sqrt{\alpha_t}\,\boldsymbol{x}_t}{1-\alpha_t}\boldsymbol{x}_{t-1} + \frac{1}{1-\bar{\alpha}_{t-1}}\boldsymbol{x}_{t-1}^2 - \frac{2\sqrt{\bar{\alpha}_{t-1}}\,\boldsymbol{x}_0}{1-\bar{\alpha}_{t-1}}\boldsymbol{x}_{t-1} + \text{const}\right]$$

Grouping the $\boldsymbol{x}_{t-1}^2$ and $\boldsymbol{x}_{t-1}$ terms gives a precision:

$$\frac{1}{\tilde{\beta}_t} = \frac{\alpha_t}{1-\alpha_t} + \frac{1}{1-\bar{\alpha}_{t-1}} = \frac{\alpha_t(1-\bar{\alpha}_{t-1}) + (1-\alpha_t)}{(1-\alpha_t)(1-\bar{\alpha}_{t-1})} = \frac{1-\bar{\alpha}_t}{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}.$$

Therefore:

$$\tilde{\beta}_t = \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t.$$

Forward posterior: the mean $\tilde{\boldsymbol{\mu}}_t$

The mean of $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$ is read off from the linear term:

$$\frac{\tilde{\boldsymbol{\mu}}_t}{\tilde{\beta}_t} = \frac{\sqrt{\alpha_t}\,\boldsymbol{x}_t}{1-\alpha_t} + \frac{\sqrt{\bar{\alpha}_{t-1}}\,\boldsymbol{x}_0}{1-\bar{\alpha}_{t-1}}.$$

Multiplying through by $\tilde{\beta}_t$:

$$\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\boldsymbol{x}_0$$

Forward posterior (exact result)

$$\boxed{q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}\big(\boldsymbol{x}_{t-1};\ \tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0),\ \tilde{\beta}_t\mathbf{I}\big)}$$

$$\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\boldsymbol{x}_0, \qquad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t$$

This is the *target* the learned reverse process must match.

VDM ELBO: full decomposition

Substituting the VDM structure into the MHVAE ELBO (Luo 2022, Sec. 3):

$$
\begin{aligned}
\log p(\boldsymbol{x}_0) \geq\; & \underbrace{\mathbb{E}_{q(\boldsymbol{x}_1|\boldsymbol{x}_0)}\big[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{x}_1)\big]}_{\mathcal{L}_0\text{: reconstruction}} - \underbrace{D_{\mathrm{KL}}\big(q(\boldsymbol{x}_T|\boldsymbol{x}_0)\,\|\,p(\boldsymbol{x}_T)\big)}_{\mathcal{L}_T\text{: prior matching}} \\
& - \underbrace{\sum_{t=2}^{T} \mathbb{E}_{q(\boldsymbol{x}_t|\boldsymbol{x}_0)}\Big[D_{\mathrm{KL}}\big(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)\,\|\,p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\big)\Big]}_{\mathcal{L}_{1:T-1}\text{: denoising matching terms}}
\end{aligned}
$$

- $\mathcal{L}_T = 0$: schedule ensures $q(\boldsymbol{x}_T|\boldsymbol{x}_0) \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- $\mathcal{L}_0$: single reconstruction step, treated separately.
- $\mathcal{L}_{1:T-1}$: dominant $T-1$ terms, each a Gaussian KL.

Why condition on $\boldsymbol{x}_0$ reduces variance

First attempt (naïve ELBO) uses $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$ in the KL, which requires an expectation over *two* random variables per term. This has high Monte Carlo variance.

Second attempt (conditioned ELBO) uses $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$. By the tower property:

$$
\begin{aligned}
&\mathbb{E}_{q(\boldsymbol{x}_{t-1},\boldsymbol{x}_t|\boldsymbol{x}_0)}\Big[D_{\mathrm{KL}}\Big(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) \,\|\, p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\Big)\Big] \\
&\quad \geq \; \mathbb{E}_{q(\boldsymbol{x}_t|\boldsymbol{x}_0)}\Big[D_{\mathrm{KL}}\Big(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\Big)\Big].
\end{aligned}
$$

The conditioned form is a tighter bound *and* requires sampling only $\boldsymbol{x}_t \sim q(\boldsymbol{x}_t|\boldsymbol{x}_0)$, which is one-shot via the closed-form marginal. Each KL term now involves only a **single Monte Carlo draw**.

Practical consequence

We compute $\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$ and evaluate the KL analytically — no expensive nested sampling.

Gaussian KL: closed-form evaluation

Each denoising term is a KL between two Gaussians. For general Gaussians $\mathcal{N}(\boldsymbol{\mu}_1, \Sigma_1)$ and $\mathcal{N}(\boldsymbol{\mu}_2, \Sigma_2)$ in $\mathbb{R}^d$:

$$D_{\mathrm{KL}}\big(\mathcal{N}(\boldsymbol{\mu}_1, \Sigma_1) \,\|\, \mathcal{N}(\boldsymbol{\mu}_2, \Sigma_2)\big) = \tfrac{1}{2}\left[\log\frac{|\Sigma_2|}{|\Sigma_1|} - d + \mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \Sigma_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)\right].$$

In our case both distributions have variance $\tilde{\beta}_t \mathbf{I}$, so the $\log|\cdot|$ and $\mathrm{tr}(\cdot)$ terms cancel exactly:

$$D_{\mathrm{KL}}\big(q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\big) = \frac{1}{2\tilde{\beta}_t}\left\|\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\right\|^2 + C$$

where $p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t), \tilde{\beta}_t \mathbf{I})$ and C does not depend on $\boldsymbol{\theta}$.

Training reduces to matching means. The network only needs to predict $\tilde{\boldsymbol{\mu}}_t$.

Reparameterising $\tilde{\boldsymbol{\mu}}_t$ in terms of noise

From the closed-form forward marginal: $\boldsymbol{x}_t = \sqrt{\bar\alpha_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon}$, we can solve for $\boldsymbol{x}_0$:

$$\boldsymbol{x}_0 = \frac{1}{\sqrt{\bar\alpha_t}}\big(\boldsymbol{x}_t - \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon}\big).$$

Substituting into $\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0)$:

$$\begin{aligned}\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{\epsilon}) &= \frac{\sqrt{\alpha_t}(1-\bar\alpha_{t-1})}{1-\bar\alpha_t}\,\boldsymbol{x}_t + \frac{\sqrt{\bar\alpha_{t-1}}\,\beta_t}{1-\bar\alpha_t}\cdot\frac{\boldsymbol{x}_t - \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon}}{\sqrt{\bar\alpha_t}} \\ &= \frac{\sqrt{\alpha_t}(1-\bar\alpha_{t-1}) + \beta_t\sqrt{\bar\alpha_{t-1}/\bar\alpha_t}}{1-\bar\alpha_t}\,\boldsymbol{x}_t - \frac{\beta_t\sqrt{\bar\alpha_{t-1}/\bar\alpha_t}\,\sqrt{1-\bar\alpha_t}}{1-\bar\alpha_t}\,\boldsymbol{\epsilon}\end{aligned}$$

Using $\bar\alpha_t = \bar\alpha_{t-1}\alpha_t$ so $\sqrt{\bar\alpha_{t-1}/\bar\alpha_t} = 1/\sqrt{\alpha_t}$, and collecting terms (see next slide):

The noise-parameterised mean

After collecting terms in $\boldsymbol{x}_t$ and $\boldsymbol{\epsilon}$:

$$\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}\right).$$

Verification. Numerator of $\boldsymbol{x}_t$ coefficient: $\alpha_t(1-\bar{\alpha}_{t-1}) + \beta_t/\alpha_t \cdot \bar{\alpha}_{t-1}/\bar{\alpha}_{t-1}$... simplifies to $(1-\bar{\alpha}_t)/\sqrt{\alpha_t}$, giving coefficient $1/\sqrt{\alpha_t}$. ✓

If the network predicts noise $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \approx \boldsymbol{\epsilon}$, then choose:

$$\boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\right).$$

The denoising KL becomes:

$$\mathcal{L}_{t-1} = \mathbb{E}_{\boldsymbol{x}_0,\boldsymbol{\epsilon}}\left[\frac{\beta_t^2}{2\tilde{\beta}_t\,\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\right\|^2\right]$$

where $\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$.

The simplified DDPM training objective

Ho et al. (2020) observed empirically that **dropping the time-dependent weighting** $\beta_t^2/(2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t))$ and averaging uniformly over t gives a simpler objective with *better* sample quality:

DDPM training loss $\mathcal{L}_{\text{simple}}$

$$\boxed{\mathcal{L}_{\text{simple}} = \mathbb{E}_{t\sim \mathrm{U}[1,T],\, \boldsymbol{x}_0,\, \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},\ t\right)\right\|^2\right]}$$

Intuition. Dropping the weight up-weights early steps (small t, small noise, easy to denoise) relative to later steps. This has a regularising effect: the network focuses more on the hard tasks where its predictions matter. The original weighted sum is still a valid ELBO; the simple loss is a reweighted ELBO that happens to work better in practice.

Three equivalent parameterisations

$\boldsymbol{x}_t, \boldsymbol{x}_0, \boldsymbol{\epsilon}$ satisfy a linear equation, so predicting any one is equivalent:

Predict	**Objective**	**Notes**
$\boldsymbol{\epsilon}$ (noise)	$\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\|^2$	DDPM; most common
$\boldsymbol{x}_0$ (data)	$\|\boldsymbol{x}_0 - \hat{\boldsymbol{x}}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\|^2$	Can over-smooth
$\boldsymbol{v} = \sqrt{\bar{\alpha}_t}\boldsymbol{\epsilon} - \sqrt{1-\bar{\alpha}_t}\boldsymbol{x}_0$	$\|\boldsymbol{v} - \boldsymbol{v}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\|^2$	Velocity; stable at $t \to 0$

Conversions at inference:

$$\hat{\boldsymbol{x}}_0 = \frac{\boldsymbol{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_{\boldsymbol{\theta}}}{\sqrt{\bar{\alpha}_t}}, \qquad \hat{\boldsymbol{\epsilon}} = \frac{\boldsymbol{x}_t - \sqrt{\bar{\alpha}_t}\,\hat{\boldsymbol{x}}_0}{\sqrt{1-\bar{\alpha}_t}}.$$

All three targets give the same denoising KL when substituted back.

The score function

The **score** of a distribution $p(\boldsymbol{x})$ is $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$. It points towards regions of high probability density.

Score of the forward-diffused distribution. Since $q(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0, (1-\bar{\alpha}_t)\mathbf{I})$, its log-density is:

$$\log q(\boldsymbol{x}_t|\boldsymbol{x}_0) = -\frac{1}{2(1-\bar{\alpha}_t)}\|\boldsymbol{x}_t - \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0\|^2 + C.$$

Differentiating with respect to $\boldsymbol{x}_t$:

$$\nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t|\boldsymbol{x}_0) = -\frac{\boldsymbol{x}_t - \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0}{1-\bar{\alpha}_t} = -\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}}.$$

The score is proportional to the *negative noise*.

DDPM as denoising score matching

Define the learned score network $\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)$. The **score matching objective** minimises:

$$\mathbb{E}_{t,\boldsymbol{x}_t}\left[\left\|\nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t) - \boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\right\|^2\right].$$

The **denoising score matching** (Vincent 2011) substitution:

$$\nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t) = \mathbb{E}_{q(\boldsymbol{x}_0|\boldsymbol{x}_t)}[\nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t|\boldsymbol{x}_0)] = -\mathbb{E}_{q(\boldsymbol{x}_0|\boldsymbol{x}_t)}\left[\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}}\right].$$

With the optimal score network $\boldsymbol{s}^*_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) = -\boldsymbol{\epsilon}^*/\sqrt{1-\bar{\alpha}_t}$:

$$\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) = -\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)}{\sqrt{1-\bar{\alpha}_t}} \implies \boldsymbol{s}^*_{\boldsymbol{\theta}} \approx \nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t).$$

Equivalence

DDPM noise prediction = denoising score matching. The DDPM network implicitly learns the score of the noisy data distribution at every noise level simultaneously.

From score to SDE: the continuous-time limit

Song et al. (2021) embed the discrete chain into a continuous SDE. As $T \to \infty$ with $\beta_t \to \beta(t)\,dt$, the forward process becomes:

$$d\boldsymbol{x} = -\tfrac{1}{2}\beta(t)\,\boldsymbol{x}\,dt + \sqrt{\beta(t)}\,d\boldsymbol{W}_t,$$

where $\boldsymbol{W}_t$ is Brownian motion. By Anderson (1982), the **reverse SDE** is:

$$d\boldsymbol{x} = \left[-\tfrac{1}{2}\beta(t)\,\boldsymbol{x} - \beta(t)\,\nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x})\right] dt + \sqrt{\beta(t)}\,d\bar{\boldsymbol{W}}_t$$

where $\bar{\boldsymbol{W}}_t$ is reverse-time Brownian motion. Replacing the score by the learned network gives:

$$d\boldsymbol{x} = \left[-\tfrac{1}{2}\beta(t)\,\boldsymbol{x} + \beta(t)\,\tfrac{\boldsymbol{\epsilon_\theta}(\boldsymbol{x},t)}{\sqrt{1-\bar{\alpha}_t}}\right] dt + \sqrt{\beta(t)}\,d\bar{\boldsymbol{W}}_t.$$

The same network $\boldsymbol{\epsilon_\theta}$ that DDPM trains is the score estimator that drives the reverse SDE.

The learned reverse process

Choose the learned reverse as:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}\big(\boldsymbol{x}_{t-1};\ \boldsymbol{\mu}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t),\ \tilde{\beta}_t \mathbf{I}\big)$$

with $\boldsymbol{\mu}_{\boldsymbol{\theta}}$ the noise-parameterised mean from Section 4.

Why use $\tilde{\beta}_t$ as the reverse variance? The true posterior variance $\tilde{\beta}_t$ lies between β_t (upper bound, for $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$) and $\tilde{\beta}_t$ (exact, for $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$). Ho et al. fixed it to $\tilde{\beta}_t$; later work (Nichol & Dhariwal 2021) learns it as an interpolation:

$$\Sigma_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) = \exp\big(v \log \beta_t + (1-v) \log \tilde{\beta}_t\big), \quad v = v_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) \in [0,1].$$

DDPM reverse step

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\right) + \sqrt{\tilde{\beta}_t}\,\boldsymbol{z}, \quad \boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})\ (t>1),\ \boldsymbol{z} = \mathbf{0}\ (t=1).$$

DDPM reverse algorithm

Complete DDPM sampling procedure

1. Sample $\boldsymbol{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. For $t = T, T-1, \ldots, 1$:
   1. $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$; $\quad \boldsymbol{z} = \mathbf{0}$ if $t = 1$.
   2. Predict noise: $\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)$.
   3. Update: $\boldsymbol{x}_{t-1} = \dfrac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \dfrac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\hat{\boldsymbol{\epsilon}}\right) + \sqrt{\tilde{\beta}_t}\,\boldsymbol{z}$.
3. Return $\boldsymbol{x}_0$.

- Each step is one network evaluation plus a cheap Gaussian update.
- With $T = 1000$: slow but high quality.
- **No discriminator**, no adversarial dynamics, no mode collapse.

DDIM: deterministic ODE sampling

The DDPM score network is *independent of the sampling procedure*, so the stochastic SDE can be replaced by a deterministic ODE:

$$d\boldsymbol{x} = \left[\frac{\boldsymbol{x} - \sqrt{1-\bar{\alpha}}\,\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}, \bar{\alpha})}{\sqrt{\bar{\alpha}}}\right] d\sqrt{\bar{\alpha}}.$$

Discretise over a subsequence $\tau_1 < \cdots < \tau_S$ $(S \ll T)$:

DDIM update $(\sigma_{\tau_i} = \eta\sqrt{\tilde{\beta}_{\tau_i}})$

$$\boldsymbol{x}_{\tau_{i-1}} = \sqrt{\bar{\alpha}_{\tau_{i-1}}}\,\hat{\boldsymbol{x}}_0 + \sqrt{1 - \bar{\alpha}_{\tau_{i-1}} - \sigma_{\tau_i}^2}\,\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_{\tau_i}, \tau_i) + \sigma_{\tau_i}\boldsymbol{z},$$

$$\hat{\boldsymbol{x}}_0 = \frac{\boldsymbol{x}_{\tau_i} - \sqrt{1-\bar{\alpha}_{\tau_i}}\,\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_{\tau_i}, \tau_i)}{\sqrt{\bar{\alpha}_{\tau_i}}}.$$

- $\eta = 0$: fully deterministic — same $\boldsymbol{x}_T$ always gives same $\boldsymbol{x}_0$.
- $\eta = 1$: recovers DDPM stochastic sampling.
- $S = 50$ matches quality of $T = 1000$ DDPM steps.

DDIM: derivation of the update rule

Start from the DDIM non-Markovian inference distribution (Song et al. 2020):

$$q_\sigma(\boldsymbol{x}_{\tau_{i-1}}|\boldsymbol{x}_{\tau_i}, \boldsymbol{x}_0) = \mathcal{N}\Big(\boldsymbol{x}_{\tau_{i-1}};\ \sqrt{\bar\alpha_{\tau_{i-1}}}\,\boldsymbol{x}_0 + \sqrt{1-\bar\alpha_{\tau_{i-1}}-\sigma^2}\ \tfrac{\boldsymbol{x}_{\tau_i}-\sqrt{\bar\alpha_{\tau_i}}\,\boldsymbol{x}_0}{\sqrt{1-\bar\alpha_{\tau_i}}},\ \sigma^2\mathbf{I}\Big).$$

Replacing $\boldsymbol{x}_0$ by the predicted $\hat{\boldsymbol{x}}_0$ and the exact noise direction by $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_{\tau_i}, \tau_i)$ recovers the update rule on the previous slide.

Why does this work? The DDIM update is a first-order ODE solver for the probability-flow ODE. The ODE and the reverse SDE share the same marginals $p_t(\boldsymbol{x}_t)$ at every time t. Since $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$ approximates the score of p_t, integrating the ODE gives the same distribution over $\boldsymbol{x}_0$ as ancestral sampling from the reverse SDE — but in far fewer steps.

Latent Diffusion Models (Stable Diffusion)

Problem. DDPM in pixel space ($512 \times 512 \times 3 = 786\,432$ dims) is too expensive.

Solution (Rombach et al. 2022): run diffusion in a compressed VAE latent space.

1. **VAE encoder**: $\boldsymbol{x} \in \mathbb{R}^{H \times W \times 3} \to \boldsymbol{z} \in \mathbb{R}^{H/8 \times W/8 \times 4}$ ($\approx 48\times$ compression).
2. **DDPM in $\boldsymbol{z}$**: much cheaper than pixel space.
3. **VAE decoder**: $\hat{\boldsymbol{z}}_0 \to \hat{\boldsymbol{x}}$.

Why this works

- VAE latent is perceptually smooth and semantically structured.
- Diffusion on $\boldsymbol{z}$ skips low-level pixel correlations.
- Conditioning (text, depth, class) via cross-attention in the U-Net.
- KL regularisation keeps $\boldsymbol{z}$ smooth enough for the diffusion prior.

VAE vs. Diffusion: mathematical parallel

Both maximise $\log p(\boldsymbol{x}) \geq \mathcal{L}(\boldsymbol{\phi}, \boldsymbol{\theta}) = \mathbb{E}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})] - D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}\|p)$. Differences lie in how this framework is applied:

Design axis	**VAE**	**DDPM**
Latent levels	1	$T = 300$–1000
Encoder	Learned $q_{\boldsymbol{\phi}}$	Fixed schedule
Bottleneck	Yes $(d_h \ll d_x)$	No $(d_h = d_x)$
Network predicts	$\boldsymbol{x}$ from $\boldsymbol{h}$	$\boldsymbol{\epsilon}$ from $(\boldsymbol{x}_t, t)$
KL terms	1	$T - 1$
Training loss	BCE + D_{KL}	MSE (noise)
Sampling cost	1 decoder call	T denoiser calls

Key insight

DDPM = VAE with (a) T latent levels, (b) fixed encoder, (c) no bottleneck.

VAE vs. Diffusion: pros and cons

VAE

Pros:

- Fast inference (1 pass)
- Compact structured latent space
- Stable training, closed-form KL
- Good for clustering / interpolation
- Scalable to large datasets

Cons:

- **Blurry samples** (ℓ_2-averaging)
- Posterior collapse risk
- ELBO gap (log-likelihood not exact)
- Mean-field limits posterior quality

Diffusion (DDPM)

Pros:

- **State-of-the-art** sample quality
- No mode collapse
- Stable training (MSE, no adversary)
- Grounded via ELBO + score matching
- Flexible conditioning

Cons:

- **Slow** sampling (T evaluations)
- No compact latent space
- High memory for large T
- Less natural for representation learning

Key equations: complete reference

Concept	Formula
Forward step	$q(\boldsymbol{x}_t \vert \boldsymbol{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\,\boldsymbol{x}_{t-1},\ (1-\alpha_t)\mathbf{I})$
Forward marginal	$q(\boldsymbol{x}_t \vert \boldsymbol{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I})$
One-shot sample	$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$
Forward posterior mean	$\tilde{\boldsymbol{\mu}}_t = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\boldsymbol{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\boldsymbol{x}_0$
Forward posterior var.	$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t$
Noise-param. mean	$\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}}\big(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}\big)$
Training loss	$\mathcal{L}_{\text{simple}} = \mathbb{E}\big[\Vert\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)\Vert^2\big]$
Score relation	$\boldsymbol{s}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t) = -\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\boldsymbol{x}_t, t)/\sqrt{1-\bar{\alpha}_t}$
Reverse step	$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\big(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\big) + \sqrt{\tilde{\beta}_t}\,\boldsymbol{z}$
SNR	$\mathrm{SNR}(t) = \bar{\alpha}_t/(1-\bar{\alpha}_t)$

Exercises

Exercises 1–3

1. **Forward marginal.** Prove $q(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\boldsymbol{x}_0, (1-\bar{\alpha}_t)\mathbf{I})$ by induction from $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$.
2. **Forward posterior.** Apply Bayes' rule to derive $q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$. Show that completing the square gives $\tilde{\boldsymbol{\mu}}_t$ and $\tilde{\beta}_t$.
3. **Gaussian KL.** Derive $D_{\mathrm{KL}}(\mathcal{N}(\boldsymbol{\mu}_1, \sigma^2\mathbf{I}) \| \mathcal{N}(\boldsymbol{\mu}_2, \sigma^2\mathbf{I})) = \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2/(2\sigma^2)$ and verify that the trace and log-det terms cancel exactly.

Exercises 4–6

4. **Noise reparameterisation.** Substitute $\boldsymbol{x}_0 = (\boldsymbol{x}_t - \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon})/\sqrt{\bar{\alpha}_t}$ into $\tilde{\boldsymbol{\mu}}_t$ and verify the result equals $\frac{1}{\sqrt{\alpha_t}}(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon})$.
5. **Score matching.** Write the score of $q(\boldsymbol{x}_t|\boldsymbol{x}_0)$ in terms of $\boldsymbol{\epsilon}$ and show the DDPM loss equals denoising score matching.
6. **ELBO decomposition.** Derive the three-term decomposition $\mathcal{L}_0 + \mathcal{L}_T + \sum_t \mathcal{L}_{t-1}$ and explain why $\mathcal{L}_T = 0$ by design.

References

1. D. Kingma & M. Welling, *Auto-Encoding Variational Bayes*, ICLR 2014. arXiv:1312.6114
2. J. Sohl-Dickstein et al., *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*, ICML 2015. arXiv:1503.03585
3. J. Ho, A. Jain & P. Abbeel, *Denoising Diffusion Probabilistic Models*, NeurIPS 2020. arXiv:2006.11239
4. J. Song, C. Meng & S. Ermon, *Denoising Diffusion Implicit Models*, ICLR 2021. arXiv:2010.02502
5. Y. Song et al., *Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR 2021. arXiv:2011.13456
6. D. Kingma et al., *Variational Diffusion Models*, NeurIPS 2021. arXiv:2107.00630
7. A. Nichol & P. Dhariwal, *Improved Denoising Diffusion Probabilistic Models*, ICML 2021. arXiv:2102.09672
8. R. Rombach et al., *High-Resolution Image Synthesis with Latent Diffusion Models*, CVPR 2022. arXiv:2112.10752
9. C. Luo, *Understanding Diffusion Models: A Unified Perspective*, 2022. arXiv:2208.11970

Transition: From Likelihood to Adversarial Training

So far we studied models maximising (a lower bound on) $\log p(\boldsymbol{x})$:

Model	Objective	Key idea
VAE	Maximise ELBO	Variational inference, 1 latent level
DDPM	Maximise ELBO (T levels)	Fixed encoder, noise prediction
GAN	**No likelihood bound**	**Adversarial game**

Fundamental difference

GANs never evaluate $\log p_{\boldsymbol{\theta}}(\boldsymbol{x})$. A **discriminator** D provides the training signal by classifying real vs. fake, driving the **generator** G to match p_r — without any likelihood computation.
Benefit: sharper samples, no ℓ_2-blurring. *Cost*: no likelihood, mode collapse risk, instability.

Generative Adversarial Networks

A **GAN** (Goodfellow et al., NeurIPS 2014) pits two neural networks against each other in a zero-sum game:

Generator G

- Input: noise $\boldsymbol{z} \sim p_z(\boldsymbol{z})$
- Output: synthetic sample $\boldsymbol{x} = G(\boldsymbol{z}; \boldsymbol{\theta}^{(g)})$
- Goal: fool the discriminator
- Implicitly defines distribution p_g

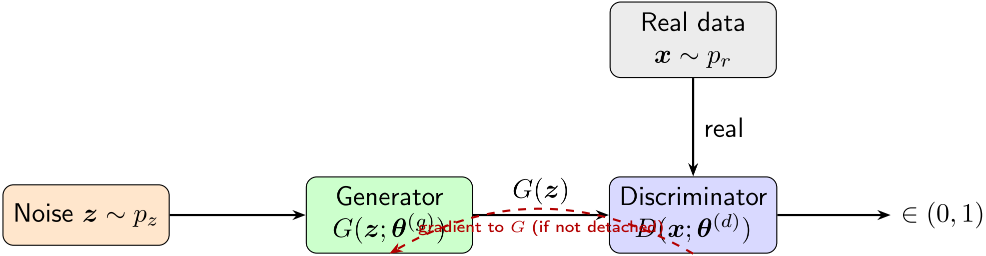
Discriminator D

- Input: image $\boldsymbol{x}$
- Output: $D(\boldsymbol{x}; \boldsymbol{\theta}^{(d)}) \in (0, 1)$ $= \Pr(\boldsymbol{x}$ is real$)$
- Goal: distinguish real from fake

Key insight

Neither network sees the other's parameters. They improve *each other* through adversarial feedback alone.

GAN Architecture (Schematic)



- G maps $\boldsymbol{z} \in \mathbb{R}^k \to \mathbb{R}^d$ (noise $\to$ data space)
- D maps $\boldsymbol{x} \in \mathbb{R}^d \to (0, 1)$ (image $\to$ probability)
- Training alternates: freeze G and update D; freeze D and update G

Learning Paradigms and Applications

Three learning settings:

1. **Unsupervised** — no labels; G learns the full data manifold
2. **Supervised** — conditional GAN conditions G on class label
3. **Semi-supervised** — D doubles as a feature extractor

Applications:

- Image synthesis / super-resolution
- Text-to-image translation
- Video prediction
- Data augmentation for rare events
- Anomaly detection
- Scientific simulation: particle physics, drug discovery, climate modelling

Notation and Probability Spaces

Symbol	Meaning
$p_r(\boldsymbol{x})$	Real data distribution (unknown; represented by training set)
$p_z(\boldsymbol{z})$	Latent noise prior, e.g. $\mathcal{N}(\mathbf{0}, I)$
$G(\boldsymbol{z}; \boldsymbol{\theta}^{(g)})$	Generator: differentiable map $\mathbb{R}^k \to \mathbb{R}^d$
p_g	Generator distribution = push-forward of p_z through G
$D(\boldsymbol{x}; \boldsymbol{\theta}^{(d)}) \in (0,1)$	Discriminator: $\Pr(\boldsymbol{x} \text{ is real})$

Implicit distribution. For invertible G the change-of-variables gives:

$$p_g(\boldsymbol{x}) = p_z\big(G^{-1}(\boldsymbol{x})\big) \left| \det \frac{\partial G^{-1}}{\partial \boldsymbol{x}} \right|.$$

In practice G is not invertible; p_g is an *implicit* distribution accessed only through sampling.

The Minimax Objective

Value function (Goodfellow et al., 2014)

$$\boxed{\min_G \max_D V(G,D) = \underbrace{\mathbb{E}_{\boldsymbol{x}\sim p_r}[\log D(\boldsymbol{x})]}_{D \text{ correct on real}} + \underbrace{\mathbb{E}_{\boldsymbol{z}\sim p_z}[\log(1 - D(G(\boldsymbol{z})))]}_{D \text{ correct on fake}}}$$

Discriminator (maximise V):

- $D(\boldsymbol{x}) \to 1$ on real data
- $D(G(\boldsymbol{z})) \to 0$ on generated data

Generator (minimise V):

- $D(G(\boldsymbol{z})) \to 1$ (fool D)
- $\mathbb{E}_{p_r}[\log D(\boldsymbol{x})]$ is constant w.r.t. G, so G minimises $\mathbb{E}[\log(1 - D(G(\boldsymbol{z})))]$

Non-Saturating Generator Loss

Problem with the original objective. Early in training D is strong, so $D(G(\boldsymbol{z})) \approx 0$:

$$\frac{\partial}{\partial \boldsymbol{\theta}^{(g)}} \log(1 - D(G(\boldsymbol{z}))) \approx \frac{\partial}{\partial \boldsymbol{\theta}^{(g)}} \log 1 = 0.$$

The gradient *vanishes* before G has learned anything.

Fix: non-saturating loss

Instead of minimising $\log(1 - D(G(\boldsymbol{z})))$, maximise $\log D(G(\boldsymbol{z}))$:

$$\mathcal{L}_G^{\text{non-sat}} = -\mathbb{E}_{\boldsymbol{z}\sim p_z}[\log D(G(\boldsymbol{z}))].$$

Same Nash equilibrium; stronger gradient when $D(G(\boldsymbol{z})) \approx 0$.

Loss form	Value at $D(G(\boldsymbol{z})) = \varepsilon \approx 0$	$\lvert$gradient$\rvert$
$\log(1-\varepsilon)$	$\approx -\varepsilon$	≈ 1 (small)
$-\log \varepsilon$	$+\infty$	$1/\varepsilon \to \infty$ (large)

The Alternating Gradient Algorithm

One training iteration

1. **Freeze** G; gradient **ascent** on D:

$$\boldsymbol{\theta}^{(d)} \leftarrow \boldsymbol{\theta}^{(d)} + \eta \, \nabla_{\boldsymbol{\theta}^{(d)}} V(G, D).$$

2. **Freeze** D; gradient **descent** on G:

$$\boldsymbol{\theta}^{(g)} \leftarrow \boldsymbol{\theta}^{(g)} - \eta \, \nabla_{\boldsymbol{\theta}^{(g)}} \mathcal{L}_G.$$

- One D step per G step in practice ($k = 1$).
- Fake images are **detached** from the computation graph during the D step to prevent spurious gradients flowing into G.
- Both networks update simultaneously — this does *not* guarantee convergence (see Section **??**).

Deriving the Optimal Discriminator

Fix G (hence fix p_g). Write V as an integral:

$$V(G,D) = \int_{\boldsymbol{x}} \big[p_r(\boldsymbol{x}) \log D(\boldsymbol{x}) + p_g(\boldsymbol{x}) \log(1 - D(\boldsymbol{x}))\big]\, d\boldsymbol{x}.$$

For each $\boldsymbol{x}$ independently, maximise the integrand $f(t) = A \log t + B \log(1-t)$, $A = p_r(\boldsymbol{x})$, $B = p_g(\boldsymbol{x})$, $t = D(\boldsymbol{x}) \in (0,1)$:

$$f'(t) = \frac{A}{t} - \frac{B}{1-t} = \frac{A - (A+B)t}{t(1-t)} = 0 \implies t^* = \frac{A}{A+B}.$$

Check: $f''(t^*) = -A/(t^*)^2 - B/(1-t^*)^2 < 0$ ✓ (maximum).

Optimal discriminator

$$\boxed{D^*(\boldsymbol{x}) = \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x}) + p_g(\boldsymbol{x})}.}$$

This is the **Bayes-optimal classifier** for the two-class problem {real, fake} under equal class priors.

Nash Equilibrium and Loss at the Optimum

Nash equilibrium. At $p_g = p_r$:

$$D^*(\boldsymbol{x}) = \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x}) + p_r(\boldsymbol{x})} = \frac{1}{2}.$$

The discriminator cannot distinguish real from fake — it is reduced to random guessing.

Loss at the equilibrium. Substituting $D^* = 1/2$:

$$V(G^*, D^*) = \log\tfrac{1}{2}\int p_r\, d\boldsymbol{x} + \log\tfrac{1}{2}\int p_r\, d\boldsymbol{x}$$
$$= -2\log 2 \approx -1.386.$$

Convergence indicator

In practice, monitor $D(\boldsymbol{x})$ and $D(G(\boldsymbol{z}))$ during training. Both should converge to 0.5 at equilibrium. If $D(G(\boldsymbol{z})) \ll 0.5$, the generator is losing. If $D(\boldsymbol{x}) \ll 0.5$, the discriminator is failing.

KL and JS Divergences

Kullback-Leibler divergence:

$$D_{\mathrm{KL}}(p\|q) = \int p(\boldsymbol{x}) \log \frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\, d\boldsymbol{x} \;\geq 0,$$

with equality iff $p = q$ a.e. (Gibbs' inequality). Note: $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$ in general.

Jensen-Shannon divergence (symmetric, bounded):

$$D_{\mathrm{JS}}(p\|q) = \frac{1}{2} D_{\mathrm{KL}}\left(p \,\Big\|\, \frac{p+q}{2}\right) + \frac{1}{2} D_{\mathrm{KL}}\left(q \,\Big\|\, \frac{p+q}{2}\right).$$

Properties of D_{JS}

- Symmetric: $D_{\mathrm{JS}}(p\|q) = D_{\mathrm{JS}}(q\|p)$
- Non-negative: $D_{\mathrm{JS}}(p\|q) \geq 0$, with equality iff $p = q$
- Bounded: $D_{\mathrm{JS}}(p\|q) \in [0, \log 2]$ for any p, q
- Related to total variation: $D_{\mathrm{JS}}(p\|q) \leq \log 2 \cdot \mathrm{TV}(p, q)$

GAN Loss = Jensen-Shannon Divergence

Expanding $D_{\mathrm{JS}}(p_r \| p_g)$ and simplifying:

$$\begin{aligned} D_{\mathrm{JS}}(p_r \| p_g) &= \frac{1}{2} D_{\mathrm{KL}}\left(p_r \,\middle\|\, \frac{p_r + p_g}{2}\right) + \frac{1}{2} D_{\mathrm{KL}}\left(p_g \,\middle\|\, \frac{p_r + p_g}{2}\right) \\ &= \frac{1}{2}\left(\log 2 + \int p_r \log \frac{p_r}{p_r + p_g}\, d\boldsymbol{x}\right) + \frac{1}{2}\left(\log 2 + \int p_g \log \frac{p_g}{p_r + p_g}\, d\boldsymbol{x}\right). \end{aligned}$$

Recognising $V(G, D^*)$ in the integrals:

Main theorem (Goodfellow et al. 2014)

$$\boxed{V(G, D^*) = 2\, D_{\mathrm{JS}}(p_r \| p_g) - 2 \log 2.}$$

Conclusion. The GAN minimax objective is equivalent to minimising the Jensen-Shannon divergence between p_r and p_g. The global minimum $-2 \log 2$ is achieved when $p_g = p_r$.

Binary Cross-Entropy Losses

Discriminator loss (real label $= s$, fake label $= 0$):

$$\mathcal{L}_D = -\frac{1}{N}\sum_{i=1}^{N}\log D(\boldsymbol{x}_i) - \frac{1}{M}\sum_{j=1}^{M}\log\big(1 - D(G(\boldsymbol{z}_j))\big).$$

Generator loss (non-saturating):

$$\mathcal{L}_G = -\frac{1}{M}\sum_{j=1}^{M}\log D(G(\boldsymbol{z}_j)).$$

One-sided label smoothing ($s = 0.9$)

Replace real target 1 with $s < 1$. The optimal discriminator becomes $D_s^*(\boldsymbol{x}) = s \cdot p_r(\boldsymbol{x})/(p_r(\boldsymbol{x}) + p_g(\boldsymbol{x}))$, preventing overconfidence and keeping gradients to G alive.

Wasserstein Distance (WGAN)

Problem with JS divergence. When p_r and p_g have non-overlapping supports (common early in training): $D_{\mathrm{JS}}(p_r \| p_g) = \log 2$ constantly, giving zero gradient everywhere.

Wasserstein-1 distance (*Earth-mover*):

$$W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(\boldsymbol{x}, \mathbf{y}) \sim \gamma} \|\boldsymbol{x} - \mathbf{y}\|$$

$\Pi(p_r, p_g)$: all couplings with marginals p_r, p_g.

Kantorovich-Rubinstein duality

$$W(p_r, p_g) = \sup_{\|f\|_L \leq 1} \left(\mathbb{E}_{p_r}[f] - \mathbb{E}_{p_g}[f]\right).$$

Sup. over all 1-Lipschitz functions f.

Advantage

W gives meaningful gradients even when $p_r \cap p_g = \emptyset$. Training is more stable.

WGAN: Gradient Penalty

WGAN critic. Replace D with a critic f_w (no sigmoid):

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_{p_g}[f_w(\boldsymbol{x})] - \mathbb{E}_{p_r}[f_w(\boldsymbol{x})] + \lambda\,\mathcal{L}_{\text{GP}}.$$

Gradient penalty (Gulrajani et al., 2017):

$$\mathcal{L}_{\text{GP}} = \mathbb{E}_{\hat{\boldsymbol{x}}}\Big[\big(\|\nabla_{\hat{\boldsymbol{x}}} f_w(\hat{\boldsymbol{x}})\|_2 - 1\big)^2\Big],$$

where $\hat{\boldsymbol{x}} = \epsilon\boldsymbol{x} + (1-\epsilon)G(\boldsymbol{z})$, $\epsilon \sim U[0,1]$ (uniform on straight lines between real and fake points).

- Enforces the 1-Lipschitz constraint via soft penalisation.
- More stable than weight clipping (original WGAN).
- $\lambda = 10$ is a standard choice.

Vanishing Gradient

When does it occur? When D is perfect early in training: $D(\boldsymbol{x}) = 1\ \forall \boldsymbol{x} \sim p_r$ and $D(G(\boldsymbol{z})) = 0\ \forall \boldsymbol{z} \sim p_z$.

Original G gradient:

$$\nabla_{\boldsymbol{\theta}^{(g)}} \mathbb{E}[\log(1 - D(G(\boldsymbol{z})))] \approx \nabla_{\boldsymbol{\theta}^{(g)}} \mathbb{E}[\log 1] = \mathbf{0}.$$

No information flows to G.

Non-saturating loss $-\log D(G(\boldsymbol{z}))$**:**

$$\nabla_{\boldsymbol{\theta}^{(g)}} \big(-\log D(G(\boldsymbol{z}))\big) = -\frac{1}{D(G(\boldsymbol{z}))} \nabla_{\boldsymbol{\theta}^{(g)}} D(G(\boldsymbol{z})).$$

When $D(G(\boldsymbol{z})) \approx 0$, the factor $1/D(G(\boldsymbol{z})) \to \infty$ amplifies the gradient — learning continues.

Mode Collapse

Definition. The generator maps many different noise vectors z to the *same* (or very few) output(s):

$$\mathrm{supp}(p_g) \ll \mathrm{supp}(p_r).$$

The JS divergence can still be low on the covered modes, so the loss does not detect collapse.

Why it happens. Given the current discriminator, G finds a single "safe" output that fools D; D then adapts, and G shifts to another single output. The two networks chase each other indefinitely.

Mitigations:

- **Minibatch discrimination**: D sees statistics of a batch, not individual images — diversity is rewarded.
- **Unrolled GANs**: G optimises against a future D, preventing short-sighted collapse.
- **Spectral normalisation**: stabilises D by bounding its Lipschitz constant.
- **WGAN**: Wasserstein distance measures full distribution mismatch, not just modes.

Non-Convergence and the Game-Theoretic View

GAN training solves a **two-player non-cooperative game**:

$$G^* = \arg\min_G \arg\max_D \, V(G, D).$$

Nash equilibrium exists at $p_g = p_r$, $D^* \equiv \frac{1}{2}$, but:

- Simultaneous gradient updates do *not* guarantee convergence to a Nash equilibrium.
- The loss landscape of each player changes at every step (non-stationary environment).
- If $\max_D V(G, D)$ is not convex in $\boldsymbol{\theta}^{(g)}$, global convergence fails even in theory.

Practical stabilisation:

- Lower β_1 in Adam (reduces momentum: 0.5 instead of 0.9)
- One-sided label smoothing
- Spectral normalisation of D's weights
- Progressive growing (ProGAN), self-attention (SAGAN)
- Two-time-scale updates: different learning rates for G and D

Evaluation Metrics: Overview

Evaluating generative models is hard: there is no ground truth output to compare against.

Metric	Measures	Limitation
Inception Score (IS)	Quality + diversity of generated images	Requires pretrained classifier
FID	Distance between real/fake feature distributions	Sensitive to sample size
MMD	Kernel-based distribution distance	Choice of kernel matters
Precision/Recall	Coverage vs. fidelity separately	Requires feature extractor

Maximum Mean Discrepancy (MMD)

Definition in RKHS. Let $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ be a kernel. The **MMD** between distributions p, q is:

$$\mathrm{MMD}^2(p,q) = \mathbb{E}_{\boldsymbol{x},\boldsymbol{x}'\sim p}[k(\boldsymbol{x},\boldsymbol{x}')] - 2\,\mathbb{E}_{\boldsymbol{x}\sim p,\,\mathbf{y}\sim q}[k(\boldsymbol{x},\mathbf{y})] + \mathbb{E}_{\mathbf{y},\mathbf{y}'\sim q}[k(\mathbf{y},\mathbf{y}')].$$

With the Gaussian kernel $k(\boldsymbol{x},\mathbf{y}) = \exp(-\|\boldsymbol{x}-\mathbf{y}\|^2/2\sigma^2)$: $\mathrm{MMD} = 0$ iff $p = q$ (characteristic kernel property).

Unbiased empirical estimator

Given n real samples $\{\boldsymbol{x}_i\}$ and m fake samples $\{\mathbf{y}_j\}$:

$$\widehat{\mathrm{MMD}}^2 = \frac{1}{n(n-1)}\sum_{i\neq j} k(\boldsymbol{x}_i,\boldsymbol{x}_j) - \frac{2}{nm}\sum_{i,j} k(\boldsymbol{x}_i,\mathbf{y}_j) + \frac{1}{m(m-1)}\sum_{i\neq j} k(\mathbf{y}_i,\mathbf{y}_j).$$

Diagonal terms ($i = j$) excluded for unbiasedness.

Fréchet Inception Distance (FID)

FID is currently the most widely used GAN metric.

Procedure:

1. Extract feature vectors from the **InceptionV3** penultimate layer for n real images and n generated images.
2. Fit Gaussians: $\mathcal{N}(\mu_r, \Sigma_r)$ and $\mathcal{N}(\mu_g, \Sigma_g)$.
3. Compute the **Fréchet distance**:

$$\mathrm{FID} = \|\mu_r - \mu_g\|^2 + \mathrm{Tr}\Big(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\Big).$$

- Lower FID = better quality and diversity.
- State-of-the-art GANs achieve FID < 5 on MNIST; < 10 on CIFAR-10.
- Sensitive to sample size n — use $\geq 10\,000$ samples.

Generator Architecture for MNIST

Fully-connected generator (Goodfellow et al., 2014 style):

$$\boldsymbol{z} \in \mathbb{R}^{100} \xrightarrow{\text{Lin+BN+LReLU}} \mathbb{R}^{256} \xrightarrow{\text{Lin+BN+LReLU}} \mathbb{R}^{512} \xrightarrow{\text{Lin+BN+LReLU}} \mathbb{R}^{1024} \xrightarrow{\text{Lin+Tanh}} \mathbb{R}^{784}$$

Design choices:

- **BatchNorm** after each Linear layer: normalises activations, allows higher LR, stabilises gradient flow
- **LeakyReLU(0.2):** prevents dying ReLU; small gradient for negative inputs
- **Tanh** output: maps to $[-1, 1]$, matching MNIST normalised to $[-1, 1]$
- Weights: $\mathcal{N}(0, 0.02^2)$ (DCGAN convention)

Discriminator differences:

- **No BatchNorm**: introduces minibatch dependencies, causes instability in D
- **Dropout(0.3)**: regularises, prevents D from over-fitting on real data
- **Sigmoid** output: probability in $(0, 1)$
- Input: flattened image $\in \mathbb{R}^{784}$

Latent Space Interpolation

A well-trained generator has a **smooth latent space**. Linear interpolation between two noise vectors:

$$\boldsymbol{z}(\alpha) = (1-\alpha)\,\boldsymbol{z}_1 + \alpha\,\boldsymbol{z}_2, \quad \alpha \in [0,1]$$

should yield a smooth visual transition between the corresponding images.

- **Smooth transitions**: generator has learned a good manifold.
- **Abrupt jumps**: mode collapse or poor latent coverage.
- Provides a qualitative test of latent space geometry that FID cannot capture.

Spherical interpolation

On a high-dimensional sphere (approximately the case for $\boldsymbol{z} \sim \mathcal{N}(0, I)$), *spherical* interpolation $\boldsymbol{z}(\alpha) = \frac{\sin((1-\alpha)\Omega)}{\sin\Omega}\boldsymbol{z}_1 + \frac{\sin(\alpha\Omega)}{\sin\Omega}\boldsymbol{z}_2$, $\cos\Omega = \hat{\boldsymbol{z}}_1 \cdot \hat{\boldsymbol{z}}_2$, stays on the noise manifold and gives more natural transitions.

Conditional GAN (cGAN)

A **conditional GAN** (Mirza & Osindero, 2014) conditions both G and D on auxiliary information $\mathbf{y}$ (class label, image, text):

$$\min_G \max_D V(G,D) = \mathbb{E}[\log D(\boldsymbol{x}|\mathbf{y})] \\ + \mathbb{E}[\log(1 - D(G(\boldsymbol{z}|\mathbf{y})))].$$

For MNIST: $\mathbf{y}$ is the digit class (one-hot, $\mathbb{R}^{10}$). Concatenate $\mathbf{y}$ to both G's input $\boldsymbol{z}$ and D's input $\boldsymbol{x}$.

cGAN enables

- Controlled generation: specify which digit to generate
- Image-to-image translation (pix2pix)
- Text-to-image synthesis
- Style transfer

Deep Convolutional GAN (DCGAN)

Radford, Metz, Chintala (ICLR 2016) replaced fully-connected layers with **strided convolutions**:

- **Generator**: fractionally strided convolutions (transposed conv) to upsample from $\boldsymbol{z} \in \mathbb{R}^{100}$ to full-resolution image.
- **Discriminator**: strided convolutions to downsample.
- BatchNorm in G (all layers except output); BatchNorm in D (except input).
- ReLU in G; LeakyReLU in D.
- No pooling layers; no fully-connected layers (except for $\boldsymbol{z}$).

Key advance

DCGAN produced the first visually convincing high-resolution GAN outputs and established the architectural conventions still used today. The DCGAN paper's guidelines (no BN in D input layer, LReLU slope 0.2, Adam with $\beta_1 = 0.5$) remain standard practice.

Comparison of GAN Variants

Variant	Key change	Loss	Advantage
Vanilla GAN	FC networks	JS div.	Simple
DCGAN	Conv. nets	JS div.	Better images
WGAN	Wt. clipping	Wasserstein	More stable
WGAN-GP	Grad. penalty	Wasser.+GP	No clipping
cGAN	Conditioning	JS or W	Controlled
StyleGAN	Style-based	W	SOTA

Key Equations: Summary

Concept	Formula
Minimax objective	$\min_G \max_D \; \mathbb{E}_{p_r}[\log D(\boldsymbol{x})] + \mathbb{E}_{p_z}[\log(1 - D(G(\boldsymbol{z})))]$
Optimal discriminator	$D^*(\boldsymbol{x}) = p_r(\boldsymbol{x})/(p_r(\boldsymbol{x}) + p_g(\boldsymbol{x}))$
Nash equilibrium	$p_g = p_r, \quad D^* \equiv 1/2$
Loss at equilibrium	$V(G^*, D^*) = -2\log 2$
GAN loss = JS div.	$V(G, D^*) = 2D_{\mathrm{JS}}(p_r \Vert p_g) - 2\log 2$
Non-sat. generator	$\mathcal{L}_G = -\mathbb{E}_{p_z}[\log D(G(\boldsymbol{z}))]$
Discriminator loss	$\mathcal{L}_D = -\mathbb{E}_{p_r}[\log D(\boldsymbol{x})] - \mathbb{E}_{p_z}[\log(1 - D(G(\boldsymbol{z})))]$
Wasserstein distance	$W(p_r, p_g) = \sup_{\Vert f \Vert_L \leq 1}(\mathbb{E}_{p_r}[f] - \mathbb{E}_{p_g}[f])$
Gradient penalty	$\lambda\, \mathbb{E}[(\Vert \nabla f_w(\hat{\boldsymbol{x}}) \Vert_2 - 1)^2]$
MMD	$\sqrt{\mathbb{E}_{p\times p}[k] - 2\mathbb{E}_{p\times q}[k] + \mathbb{E}_{q\times q}[k]}$
FID	$\Vert \mu_r - \mu_g \Vert^2 + \mathrm{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$

Exercises

1. **Optimal discriminator.** Derive $D^*(\boldsymbol{x}) = p_r/(p_r + p_g)$ by differentiating $f(t) = A \log t + B \log(1-t)$ and verify it is a maximum, not a minimum.
2. **JS divergence bounds.** Prove $D_{\text{JS}}(p\|q) \in [0, \log 2]$ for any two distributions and find the conditions for equality on each side.
3. **Vanishing gradient.** At $D(G(\boldsymbol{z})) = \varepsilon \ll 1$, compute $|\nabla_{\boldsymbol{\theta}^{(g)}} \mathcal{L}_G^{\text{orig}}|$ and $|\nabla_{\boldsymbol{\theta}^{(g)}} \mathcal{L}_G^{\text{non-sat}}|$ and compare their magnitudes as $\varepsilon \to 0$.
4. **Label smoothing.** Derive the optimal discriminator $D_s^*(\boldsymbol{x})$ when real targets are smoothed to $s \in (0,1)$ instead of 1. Show it converges to D^* as $s \to 1$.
5. **Wasserstein distance.** Compute $W(\mathcal{N}(0,1), \mathcal{N}(\mu,1))$ analytically and verify that $W \to 0$ as $\mu \to 0$, unlike D_{JS} when the two Gaussians have non-overlapping support.
6. **Conditional GAN.** Extend the minimax objective to the conditional setting. What changes in the derivation of $D^*(\boldsymbol{x}|\mathbf{y})$?

Three Families: A Unified Mathematical View

All three learn $p_{\boldsymbol{\theta}}(\boldsymbol{x})$ but with different objectives and inductive biases.

Criterion	**VAE**	**Diffusion**	**GAN**
Objective	ELBO	ELBO (T terms)	Minimax / JS
Likelihood	$\checkmark$	$\checkmark$	$\times$
Latent space	Compact $\boldsymbol{h}$	Full-dim $\boldsymbol{x}_t$	Noise $\boldsymbol{z}$
Encoder	Learned	Fixed schedule	None
Sampling cost	1 decoder pass	T steps	1 pass
Stability	High	High	Low–Moderate
Sample quality	Moderate	State of the art	High (sharp)
Mode coverage	Good	Excellent	Partial
Anomaly detect.	$\checkmark$	$\checkmark$	$\times$

Why VAEs Produce Blurry Samples

Mathematical reason. The VAE decoder minimises:

$$-\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{h})\right] \;\propto\; \mathbb{E}_{q_{\boldsymbol{\phi}}}\left[\|\boldsymbol{x} - \hat{\boldsymbol{x}}_{\boldsymbol{\theta}}(\boldsymbol{h})\|^2\right].$$

Because the posterior $q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})$ has nonzero variance, the decoder learns the **posterior mean over all likely completions**, which is a blur over the data manifold.

Contrast with the other two models:

Diffusion avoids blurring by

predicting only the *noise added at one step* t, not the full image. Each reverse step makes a small, committed correction. The iterative nature removes the need to average over the full distribution of plausible outputs.

GANs avoid blurring by

replacing the fixed ℓ_2 loss with an *adaptive, learned* discriminator loss. The discriminator penalises blurriness and any systematic deviation from real data, providing a richer training signal than any fixed pixel-wise objective.

The Likelihood Problem in GANs

A fundamental asymmetry. VAEs and diffusion models optimise a lower bound on $\log p(\boldsymbol{x})$, so they can both *generate* and *evaluate* density. GANs have **no access to** $p_g(\boldsymbol{x})$.

Consequences of no likelihood:

- Cannot compute log-likelihood scores for anomaly detection.
- Cannot evaluate model fit on held-out data in a principled way.
- Evaluation requires proxy metrics (FID, IS, MMD) rather than test-set log-likelihood.
- Cannot be embedded as a prior in a Bayesian framework without approximation.

Why this is a price worth paying (sometimes). The discriminator provides a *perceptual* loss that no fixed $p_{\boldsymbol{\theta}}$ objective can replicate: it adapts to what "looks real" to a neural network, capturing texture, sharpness, and high-frequency detail that ℓ_2 and even ELBO objectives systematically underweight.

Training Stability: A Comparison

VAE — Stable

ELBO is a well-defined scalar. Both networks trained with standard SGD. KL prevents collapse when weighted correctly. Main challenge: choosing β for β-VAE.

Diffusion — Stable

MSE on noise prediction is smooth. No adversarial dynamics. Low MC gradient variance ($\boldsymbol{x}_t$ sampled in one step). Main challenge: noise schedule design.

GAN — Unstable

Non-stationary minimax landscape. No Nash convergence guarantee. $V(G, D)$ is concave-convex; simultaneous gradient ascent-descent diverges even for bilinear games. Requires WGAN-GP, spectral norm, label smoothing.

Key insight

GAN instability is *fundamental*: the minimax saddle structure prevents convergence guarantees that both VAE and DDPM enjoy.

Mode Collapse vs. Posterior Averaging: Two Failure Modes

The two pathological failure modes of generative models lie at opposite ends:

Posterior averaging (VAE)

Too much spread. The decoder must explain all data through a single mean. Result: generated images are averages over the data manifold — blurry. The model *covers* all modes but fails to commit to any one of them.

$$\hat{\boldsymbol{x}} = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{h}|\boldsymbol{x})}[\hat{\boldsymbol{x}}_{\boldsymbol{\theta}}(\boldsymbol{h})] \approx \text{ mean of modes}$$

Mode collapse (GAN)

Too little spread. The generator finds a single safe output that fools D and ignores the rest of p_r. Result: sharp images but only from a few modes. $\mathrm{supp}(p_g) \ll \mathrm{supp}(p_r)$.

$$G(\boldsymbol{z}) \approx \boldsymbol{x}^* \ \forall \boldsymbol{z} \ \Rightarrow \ D_{\mathrm{JS}}(p_r \| p_g) \approx \log 2 \text{ (flat)}$$

Diffusion avoids both: the fixed encoder $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$ prevents collapse (no network can "cheat" the noise schedule), and the iterative denoising avoids averaging because each step commits to only a small noise correction rather than a full image prediction.

Detailed Pros and Cons: VAEs

Pros

- **Likelihood bound**: ELBO usable for anomaly detection.
- **Compact latent**: $\boldsymbol{h}$ supports interpolation, clustering, disentanglement, property prediction.
- **Fast generation**: single decoder pass.
- **Stable training**: well-defined ELBO, no adversary.
- **Flexible**: β-VAE for disentanglement; hierarchical VAEs for expressivity.
- **Scientific use**: ideal for molecules, spectra, simulations.

Cons

- **Blurry samples**: posterior averaging is fundamental, not a capacity issue.
- **Posterior collapse**: decoder ignores $\boldsymbol{h}$; KL$\to 0$, latent useless.
- **ELBO gap**: $\log p(\boldsymbol{x}) = \mathrm{ELBO} + D_{\mathrm{KL}}(q_\phi \| p(\boldsymbol{h}|\boldsymbol{x})) > \mathrm{ELBO}$.
- **Mean-field**: factorised Gaussian misses correlated posteriors.
- **Fixed-dim bottleneck**: wrong d_h hurts quality.

Detailed Pros and Cons: Diffusion Models

Pros

- **Best sample quality**: sharp, diverse, no blurring.
- **Full coverage**: fixed forward process prevents mode collapse.
- **Stable training**: MSE on noise, no adversarial dynamics.
- **Likelihood bound**: ELBO $\to \log p(\boldsymbol{x})$ as $T \to \infty$.
- **Flexible conditioning**: text, class, depth via classifier-free guidance and cross-attention.
- **Rich theory**: SDE/score connection (Song et al. 2021).

Cons

- **Slow sampling**: $T = 100$–1000 steps (DDIM/DPM-Solver: 10–50).
- **No compact latent**: full-dim $\boldsymbol{x}_t$ precludes clustering or property prediction.
- **Compute-intensive**: large T, high-dim $\boldsymbol{x}$ need significant GPU memory.
- **Poor for repr. learning**: no encoder for features.
- **Schedule-sensitive**: cosine preferred over linear at high resolution.

Detailed Pros and Cons: GANs

Pros

- **Sharpest samples**: adversarial loss catches blurriness that ℓ_2/ELBO miss.
- **Fastest inference**: single generator pass.
- **Flexible loss**: works when likelihood is intractable.
- **Rich ecosystem**: cGAN, DCGAN, WGAN-GP, StyleGAN, pix2pix.
- **Image translation**: handles paired and unpaired transfer.

Cons

- **No likelihood**: no anomaly detection or density estimation.
- **Mode collapse**: G covers only part of p_r; FID/IS may not detect it.
- **Instability**: non-stationary minimax; no convergence guarantee; hyperparameter-sensitive.
- **JS flatness**: $D_{\mathrm{JS}} = \log 2$ when supports don't overlap (WGAN fixes this).
- **Wasted discriminator**: D discarded after training.

When to Use Each Model

Use case	Model	Reason
Image / audio / video synthesis	Diffusion	Best quality, no collapse
Text-to-image (Stable Diffusion)	LDM	VAE compression + diffusion
Image-to-image translation	GAN	Adversarial perceptual loss
Real-time generation	VAE or GAN	Single-pass inference
Anomaly detection	VAE or Diffusion	Likelihood bound available
Representation / clustering	VAE	Compact structured latent
Scientific data (molecules)	VAE	Low-dim latent for prediction
Data augmentation	GAN or Diffusion	Sharp, diverse samples
Inpainting / super-resolution	Diffusion	Score-based posterior
Very limited data	VAE	Stable; KL regularisation

Key Equations: Complete Summary Across All Three Models

Concept	VAE / Diffusion	GAN
Encoder	$q_{\boldsymbol{\phi}}(\boldsymbol{h}\|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}, \boldsymbol{\sigma}_{\phi}^2\mathbf{I})$ $q(\boldsymbol{x}_t\|\boldsymbol{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\boldsymbol{x}_{t-1}, (1-\alpha_t)\mathbf{I})$	None (implicit via G)
Marginal / forward	$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$	$\boldsymbol{x} = G(\boldsymbol{z}; \boldsymbol{\theta}^{(g)})$
Objective	$\mathcal{L} = \mathbb{E}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}\|\boldsymbol{h})] - D_{\mathrm{KL}}(q\\|p)$	$\min_G \max_D \mathbb{E}[\log D(\boldsymbol{x})] + \mathbb{E}[\log(1 - D(G(\boldsymbol{z})))]$
Optimal solution	DDPM: $\boldsymbol{\mu}_{\boldsymbol{\theta}} = \frac{1}{\sqrt{\alpha_t}}(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_{\boldsymbol{\theta}})$	$D^*(\boldsymbol{x}) = \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x})+p_g(\boldsymbol{x})}$
Loss at equilibrium	VAE ELBO $\to \log p(\boldsymbol{x})$	$V(G^*, D^*) = -2\log 2$
Divergence minimised	$D_{\mathrm{KL}}(q\\|p)$ at each level	$D_{\mathrm{JS}}(p_r\\|p_g)$
Score connection	$\boldsymbol{s}_{\boldsymbol{\theta}} = -\boldsymbol{\epsilon}_{\boldsymbol{\theta}}/\sqrt{1-\bar{\alpha}_t}$	$D^* \to$ implicit score estimator
Sampling	1 pass (VAE); T steps (DDPM)	1 pass

Exercises: Cross-Model Comparisons

Exercises 1–3

1. **Common ELBO.** Show VAE (1 level) and DDPM (T levels) are special cases of the MHVAE ELBO. Identify encoder, decoder, prior in each.
2. **Blurriness from ℓ_2.** For $p_r = \frac{1}{2}\delta(x-a) + \frac{1}{2}\delta(x-b)$, show the VAE decoder learns $\hat{x} = (a+b)/2$ while a GAN can produce both modes.
3. **Convergence.** State which property of the DDPM ELBO makes it amenable to SGD while the GAN minimax saddle is not.

Exercises 4–6

4. **Wasserstein vs. JS.** Compute $D_{\mathrm{JS}}(\mathcal{N}(0,1)\|\mathcal{N}(\mu,1))$ and $W(\mathcal{N}(0,1),\mathcal{N}(\mu,1))$. Show D_{JS} saturates at $\log 2$ while $W = |\mu|$ grows.
5. **Score matching bridge.** Using $\nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t|\boldsymbol{x}_0) = -\boldsymbol{\epsilon}/\sqrt{1-\bar{\alpha}_t}$, show DDPM equals denoising score matching (Vincent 2011).
6. **Latent Diffusion.** Express the LDM objective as a composition of VAE ELBO and DDPM ELBO. Identify which parameters are trained in each stage.

References

Diffusion models

1. Kingma & Welling, *Auto-Encoding Variational Bayes*, ICLR 2014. 1312.6114
2. Ho, Jain & Abbeel, *DDPM*, NeurIPS 2020. 2006.11239
3. Song et al., *Score-Based Generative Modeling through SDEs*, ICLR 2021. 2011.13456
4. Song, Meng & Ermon, *DDIM*, ICLR 2021. 2010.02502
5. Kingma et al., *Variational Diffusion Models*, NeurIPS 2021. 2107.00630
6. Nichol & Dhariwal, *Improved DDPM*, ICML 2021. 2102.09672
7. Rombach et al., *Latent Diffusion Models*, CVPR 2022. 2112.10752
8. Luo, *Understanding Diffusion Models*, 2022. 2208.11970

Generative Adversarial Networks

1. Goodfellow et al., *Generative Adversarial Nets*, NeurIPS 2014. 1406.2661
2. Mirza & Osindero, *Conditional GAN*, 2014. 1411.1784
3. Radford, Metz & Chintala, *DCGAN*, ICLR 2016. 1511.06434
4. Arjovsky, Chintala & Bottou, *WGAN*, ICML 2017. 1701.07875
5. Gulrajani et al., *WGAN-GP*, NeurIPS 2017. 1704.00028
6. Heusel et al., *FID*, NeurIPS 2017. 1706.08500
7. Goodfellow, Bengio & Courville, *Deep Learning*, MIT Press 2016, Ch. 20.
8. Weng, *From GAN to WGAN*, blog post, 2017.